

Data Understanding Notebook

Descriptive Statistics

by Lorraine Gaudio

Version May 29, 2026

Contents

1	Overview	2
2	Set Up Data Understanding Notebook	3
2.1	Task 1	3
2.2	Task 2	5
2.3	Task 3	6
3	Understand the Data Before Summarizing	7
3.1	Variables and Data Types	7
4	Practice Space 1	8
4.1	Task 4	9
4.2	Task 5	9
5	Count-Based Summaries	10
5.1	Length	11
5.2	Count and Mode	11
6	Practice Space 2	12
6.1	Task 6	12
7	Measures of Center for Numerical Data	14
7.1	Median	14
8	Practice Space 3	16
8.1	Task 7	16
8.2	Mean	17
9	Practice Space 4	19
9.1	Task 8	20
9.2	Task 9	20
10	Measures of Spread	22
10.1	Standard Deviation	22
11	Practice Space 5	25
11.1	Task 10	25
11.2	Task 11	27
11.3	Task 12	27
11.4	Range	29

12 Practice Space 6	30
12.1 Task 13	30
12.2 Interquartile Range (IQR)	32
13 Practice Space	33
13.1 Task 14	33
13.2 Five-Number Summary	35
14 Practice Space	36
14.1 Task 15	36
14.2 Task 16	37
15 Summary	38
16 Summary	38
17 Chapter Terms	38

1 Overview

This Signature Assignment asks you to build a **Data Understanding Notebook** in Quarto using the **penguins** dataset. The notebook is a working analytical document where you load data, identify variable types, choose appropriate descriptive statistics, and explain what those statistics show. This activity asks you to plan your approach, monitor whether your code and interpretations make sense, and revise your thinking when something is unclear or incorrect. That process is part of self-regulated learning: using metacognitive strategies to guide your own understanding.

This activity is also designed to help you practice writing words around code. In this course, explaining your code and output in plain English is a learning strategy that helps you rehearse key ideas, strengthen your memory, and check whether you truly understand what the code is doing. Each part builds on previous sections, so you return to earlier ideas and use them again in a new context. This repetition helps you develop fluency.

Learning how to create a **Data Understanding Notebook** is something you can use in real analytical settings. Researchers do not begin with conclusions. They begin by getting to know the data: what each variable represents, what kind of values it contains, which summaries make sense, and what early patterns or problems deserve attention. This notebook reflects that real workflow. It helps you practice persistence on difficult tasks, document your thinking, and build habits that matter in real analytical settings where clear reasoning, reproducible work, and thoughtful interpretation are expected.

This activity is designed to be open-ended. You have the freedom to choose which variables to describe and which descriptive statistics to use. The goal is not to get a “right answer” but to make informed choices and explain your reasoning. You will learn how to connect the reading to the data, make decisions about which summaries fit which kinds of variables, and explain why those summaries make sense.

By the end of the Descriptive Statistics activity you will be able to:

- Define key data science terms related to data collection and data types.
- Calculate and interpret common descriptive statistics such as counts, mode, median, mean, variance, and standard deviation.
- Report descriptive statistics using inline R code in a Quarto document.
- Understand the importance of choosing appropriate descriptive statistics based on the type of data and research question.
- Produce a well-structured and formatted HTML document summarizing the descriptive statistics of a dataset.

2 Set Up Data Understanding Notebook

This section builds the foundation for your **Data Understanding Notebook**. You will create a Quarto file, work in the **Source** editor, set up the document header, and add the first sections of your report.

You are still new to Quarto, so move slowly. The goal is not just to make a file. The goal is to understand how Quarto formatting becomes a clean HTML report.

2.1 Task 1

Goal: Set up notebook

2.1.1 Step 1

1. Open RStudio.
2. Create a new Quarto document:

File > New File > Quarto Document

3. Enter the document title and your name.
 - Title: **Data Understanding Notebook**
 - Author: **YOUR NAME**
4. Choose the output format as HTML.
5. Uncheck the option to use visual markdown editor.

The **Visual** editor can be useful later, but it may hide some of the formatting marks you need to practice in this activity. Make sure you are working in the **Source** editor.

In the **Source** editor, you can see the Quarto formatting symbols you are learning, including:

- hashtags for headings, such as **# Purpose of this notebook**
 - backticks for code, such as **penguins**
 - code chunk fences, such as **```{r}**
6. Select **Create Empty Document** to start with a blank slate.

2.1.2 Step 2

Modify your YAML header to customize your Quarto document. The YAML header is the section at the very top of the file between the two lines of three dashes: ---.

The YAML header controls information about the document and how the HTML report will be created.

Replace the YAML header at the top of your Quarto file with this:

```
---
title: "Data Understanding Notebook"
subtitle: "Descriptive Statistics"
author: "YOUR NAME"
date: today
toc: true
toc-depth: 2
number-sections: true
format: html
---
```

Here is what these YAML settings do:

- **title** gives the report its main title.

Step 1: Title & purpose

New Quarto Document

Document
Presentation
Interactive

Title:

Author:

☒ **HTML**
Recommended format for authoring (you can switch to PDF or Word output anytime)

☐ **PDF**
PDF output requires a LaTeX installation (e.g. <https://yihui.org/tinytex/>)

☐ **Word**
Previewing Word documents requires an installation of MS Word (or Libre/Open Office on Linux)

Engine:

Editor: ☐ Use visual markdown editor ?

? [Learn more about Quarto](#)

Create Empty Document Create Cancel

Figure 1: New Quarto Document Form

- `subtitle` adds a smaller title underneath the main title.
- `author` identifies who created the notebook.
- `date: today` tells Quarto to use the current date when the document renders.
- `toc: true` creates a table of contents.
- `toc-depth: 2` includes level 1 and level 2 headings in the table of contents.
- `number-sections: true` numbers the report sections.
- `format: html` tells Quarto to create an HTML report.

Save the file as `data_understanding_notebook.qmd` in your course folder.

2.1.3 Step 3

Add only the headings you need for this first section.

Copy and paste these headings into your Quarto file below the YAML header:

```
# Purpose of this notebook

# Load Packages and Data
```

You will add more sections later as the notebook grows.

Tip: Render to HTML often. Rendering helps you check whether your Quarto formatting is working the way you expect.

2.2 Task 2

Add a notebook purpose statement

Goal: Start your Data Understanding Notebook with a purpose statement.

In this task, you do not need to write your own purpose statement. Copy the provided paragraph so you can focus on learning where this kind of statement belongs in a Quarto report.

2.2.1 Step 1

Under the heading `# Purpose of this notebook`, copy and paste this paragraph into your Quarto file. *This paragraph is report text. It does not go inside an R code chunk.*

This document is an exploratory data analysis notebook used to understand the ``penguins`` dataset and pr

Notice:

- `penguins` has backticks on each side of it because it is the name of an R object. The backticks tell Quarto to format it as **inline code** in the final report.
- The research question uses bold formatting because it is the central question for the notebook.
- The paragraph explains the dataset, the purpose of the notebook, and the guiding question.

Notice: backticks are located on the same key as the tilde ~ on most keyboards. This is usually above the tab key. You use three backticks to start and end an R code chunk. You will use one backtick on each side of inline code.

2.2.2 Step 2

Add a link to the dataset reference page.

Copy and paste this Markdown link under the purpose statement:

`[Dataset References](https://allisonhorst.github.io/palmerpenguins/reference/penguins.html)`

This creates a clickable hyperlink in your rendered HTML report.

It will look like this:

Dataset References

2.3 Task 3

Load packages and data

Under the heading `# Load Packages and Data`, you will add report text and R code.

This section teaches two parts of a reproducible report:

- regular text that explains what the section does
- R code chunks that load the packages and data

2.3.1 Step 1

First, add a short goal statement under `# Load Packages and Data`.

Copy and paste this sentence into your Quarto file:

```
** Goal:** Load the packages and data needed for the Data Understanding Notebook. I will use the palmerpenguins package.
```

*This sentence is report text. It does **not** go inside an R code chunk.*

The package names use backticks because they are R package names.

2.3.2 Step 2

Next, add a note with the package reference link.

Copy and paste this sentence under the goal statement:

```
Package reference: <https://education.rstudio.com/blog/2020/07/palmerpenguins-cran/>
```

This gives you a citation link you can return to later.

2.3.3 Step 3

Add this R code chunk to install packages if needed.

You only need to install a package once on your computer. That is why the install lines are commented out with `#`.

```
# Install once if needed:  
#| eval: false  
install.packages("dplyr")  
install.packages("palmerpenguins")
```

Notice: In Quarto, every code chunk is run when you render the report. That means `install.packages()` will try to install packages during the render. This can cause the HTML render to fail, especially if Quarto needs permission, an internet connection, or a clean setup.

Use `#| eval: false` for install lines so they appear in the report but do not run during rendering.

2.3.4 Step 4

Add this R code chunk to load the packages.

```
#
library(dplyr)
library(palmerpenguins)
```

The chunk options `message=FALSE` and `warning=FALSE` keep package startup messages out of the final HTML report.

2.3.5 Step 5

Add this R code chunk to check that the `palmerpenguins` package data are available.

```
#
data(package = "palmerpenguins")
```

2.3.6 Step 6

Add a short comment under the code chunks.

```
**Comment:** The `palmerpenguins` package provides the dataset for this notebook. The `dplyr` package g
```

Tip: Render your document to HTML to check how your report looks before moving on. Look for and correct any formatting issues. This also ensures that your report renders without errors before you add more sections.

You have now started the report itself. The next section works differently.

You are about to read background information that will help you make better choices in the next report-building tasks. Do **not** copy the reading section into your Quarto report. Use it as a guide for deciding what to add later.

3 Understand the Data Before Summarizing

This section begins with a reading. Pause report-building for a moment before you continue building your Data Understanding Notebook.

The reading will help you understand what variables are, what different data types mean, and why different kinds of variables need different kinds of summaries.

Skim the reading, paying closer attention to the report formatting than the statistical concepts. Then, you'll return to your Quarto file to complete tasks that use the `penguins` dataset. You will do that in a new report section called `# Variables and Data Types`.

3.1 Variables and Data Types

1. **Unordered sample:** A sample where the order of the elements does not matter. For example, if we have a population of 5 elements and we take a sample of 3, the set `{1, 2, 3}` is the same as `{3, 2, 1}`.

Create the vector `heights_sample` to store the set values: `5.9ft`, `6.1ft`, `5.8ft`, `6.0ft`, `5.7ft`. This is a sample of heights of 5 *individuals* from a population of all individuals in a city.

```
<!-- Example in Quarto -->
Create the vector `heights_sample` to store the set values: $5.9ft, $6.1ft, $5.8ft, $6.0ft, $5.7ft

#
heights_sample <- c(5.9, 6.1, 5.8, 6.0, 5.7) # heights in feet
```

```
<!-- Example of inline R code in Quarto -->
```

```
The sample contains {r} length(heights_sample) height measurements. The order has no inherent meaning.
```

This is what it looks like after rendering:

The sample contains 5 height measurements. The order has no inherent meaning.

2. **Ordered sample:** A sample where the order of the elements does matter. Time series data, growth measurements, finishing positions in a race, and data sorted from smallest to largest for calculating quantiles or medians are examples of ordered samples.

If we take 5 samples of the height of *one* youth as they grow up, we have an ordered sample.

```
#
```

```
growth_data <- c(42, 45, 48, 51, 54) # Heights in inches by age
```

```
<!-- Example of inline R code in Quarto -->
```

```
The ordered sample contains {r} length(growth_data) height measurements. The first value is {r} growth_data[1].
```

This is what it looks like after rendering:

The ordered sample contains 5 height measurements. The first value is 42 inches, and the last value is 54 inches.

The order of the values matters because they represent measurements taken at different ages. The first value corresponds to the height at the youngest age, and the last value corresponds to the height at the oldest age. The set {42, 45, 48, 51, 54} is **NOT** the same as {45, 54, 48, 42, 51}.

Data that can be divided into categories or groups is called **categorical data**. For example, the colors of cars in a parking lot are categorical data. In R, categorical data is often represented as factors. Factors can be ordered or unordered.

A list of placements in a race is an ordered sample because the order of the placements matters. The first-place finisher is different from the second-place finisher, and so on.

```
#
```

```
race_positions <- c("Kim", "Alex", "Jordan", "Taylor") # Order indicates ranking
```

```
<!-- Example of inline R code in Quarto -->
```

```
The race data include {r} length(race_positions) finishers. {r} race_positions[1] finished first.
```

This is what it looks like after rendering:

The race data include 4 finishers. Kim finished first.

The set {Kim, Alex, Jordan, Taylor} is **NOT** the same as {Taylor, Jordan, Alex, Kim}.

4 Practice Space 1

Now return to your Quarto file.

In this Practice Space, you will add a new report section called **# Variables and Data Types**. This section connects the reading to the **penguins** dataset.

You will inspect the dataset, report the number of rows and columns, identify the R class of each variable, and explain why variable type matters when choosing descriptive statistics.

Add this heading to your Quarto file:

Variables and Data Types

4.1 Task 4

Create a dataset overview

Under # Variables and Data Types, start by checking the structure of the `penguins` dataset.

Goal: Inspect the `penguins` dataset and report the number of rows and columns in the Data Understanding Notebook.

```
#
glimpse(penguins)

## Rows: 344
## Columns: 8
## $ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
## $ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
## $ bill_length_mm <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ~
## $ bill_depth_mm <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ~
## $ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186~
## $ body_mass_g   <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ~
## $ sex          <fct> male, female, female, NA, female, male, female, male~
## $ year         <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~

colnames(penguins)

## [1] "species"      "island"        "bill_length_mm"
## [4] "bill_depth_mm" "flipper_length_mm" "body_mass_g"
## [7] "sex"          "year"
```

Checks: Look carefully at the output from `glimpse()`. Find the number of rows, the number of columns, the column names, and the class of each variable.

Tip: In the `glimpse()` output, `<fct>` means factor, `<dbl>` means double, and `<int>` means integer.

Now add an inline R sentence to your report.

Copy and paste this sentence below the code chunk:

```
**Comment:** The `penguins` dataset contains `{r} nrow(penguins)` rows and `{r} ncol(penguins)` columns.
```

Render to HTML to see the result.

It should look like this:

Comment: The `penguins` dataset contains 344 rows and 8 columns.

If your rendered sentence still shows the code instead of the numbers, check that you typed the inline R code with one backtick on each side.

4.2 Task 5

State class and variable type

In this task, you will describe the variables in the `penguins` dataset.

You will report:

1. the variable name
2. the R class of the variable
3. whether the variable is categorical or numerical

4. if numerical, whether it is discrete or continuous
5. whether the order of the values matters

Goal: Use inline R code to describe the R class and data type for each variable in the `penguins` dataset.

First, use these groups to help decide which sentence template to use.

Copy the sentence templates below into your report. Replace each `_____` with the correct column name.

Template A: factor / categorical / unordered

Comment: The `_____` column is stored as `{r} class(penguins$_____)` in R. This is **categorical**

Template B: double / numerical / continuous / unordered

Comment: The `_____` column is stored as `{r} class(penguins$_____)` in R. This is **numerical**

Template C: integer / numerical / continuous / unordered

Comment: The `_____` column is stored as `{r} class(penguins$_____)` in R. This is **numerical**

Template D: integer year / numerical / discrete / ordered

Comment: The `_____` column is stored as `{r} class(penguins$_____)` in R. This is **numerical**

Checks: Render to HTML and confirm that each sentence shows the R class output, such as `factor`, `numeric`, or `integer`, instead of showing the inline R code.

Reflection: Was there anything surprising about the R classes in the `penguins` dataset?

Sometimes variables have an R class that needs careful interpretation. For example, `flipper_length_mm` and `body_mass_g` are stored as integers, but they still represent measured body characteristics. The `year` column is also stored as an integer, but it represents time rather than a body measurement.

These classes are not wrong, but they require interpretation. Checking the structure of your data helps you avoid choosing the wrong descriptive statistics later. For example, you would not want to calculate the mean of the `year` column because it does not represent a body measurement. You would also want to be careful about calculating the mean of the `sex` column because it is categorical data that is stored as a factor. The mean of a factor does not make sense.

5 Count-Based Summaries

Pause report-building for a moment while you review the reading section before you continue building your Data Understanding Notebook.

This reading section explains what count-based summaries are, how to calculate them, and when they are appropriate. Skim the reading, focusing more on the report formatting than the statistical concepts. The statistical concepts will help you decide which variables in the `penguins` dataset make sense for count-based summaries.

After the reading, you will return to your Quarto file and add a new report section called `# Count-Based Summaries`. Here, you will practice writing inline R code to report the number of values in a column, create frequency tables with `count()`, and identify the mode.

5.1 Length

Length counts the number of values in a set. In other words, it tells us how many observations we have. In a data table, length can tell us how many values are in one column.

Goal: Count the number of values in the `mpg` column of the `mtcars` dataset.

```
#  
n <- length(mtcars$mpg)  
  
<!-- Example of inline R code in Quarto -->  
**Comment:** The `mpg` column in the `mtcars` dataset contains `{r} length(mtcars$mpg)` values.
```

This is what it looks like after rendering:

Comment: The `mpg` column in the `mtcars` dataset contains 32 values.

You can also use `nrow(mtcars)` to get the number of rows in the dataset. This gives the same result as `length(mtcars$mpg)` because each row has one value for `mpg`.

```
<!-- Example of inline R code in Quarto -->  
**Comment:** The `mtcars` dataset contains `{r} nrow(mtcars)` rows.
```

This is what it looks like after rendering:

Comment: The `mtcars` dataset contains 32 rows.

In Chapter 13, you will work inside `summarize()`. When you are summarizing rows inside groups, you will often use `n()` to count the number of rows in each group.

5.2 Count and Mode

When you want to know how often each value appears in a column, use the `dplyr` function `count()`.

This is especially useful for categorical variables and discrete variables. A frequency table shows how many times each unique value appears.

The **mode** is the value that appears most often. Once you create a frequency table, the mode is the value with the largest count.

Syntax:

```
data %>%  
  count(variable)
```

Use `?count` to learn more about the function.

Goal: Use `count()` to create a frequency table for the `cyl` column of the `mtcars` dataset. Then determine the mode.

```
#  
cyl_counts <- mtcars %>%  
  count(cyl)  
  
#  
cyl_counts
```

```
##   cyl  n  
## 1    4 11  
## 2    6  7  
## 3    8 14
```

Comment: In this table, `n` is the number of times each unique value appears in the `cyl` column. Cars with 8 cylinders occur most frequently, so 8 is the mode for `cyl` in the `mtcars` dataset.

Note: If no value occurs more than once, then there may be no single useful mode. In some classes, you may hear that every value is a mode. In practice, it is usually better to say there is **no clear mode**.

6 Practice Space 2

Now return to your Quarto file.

In this Practice Space, you will add a new report section called `# Count-Based Summaries`. This section connects the reading to the `penguins` dataset.

Before you begin, check the difference between these three counting tools:

- `length()` counts the number of values in a column.
- `sum()` adds up numeric values in a column.
- `count()` counts how often each unique value appears.

For example, with a variable like `cyl`, we usually care about how many cars have 4, 6, or 8 cylinders. That is a `count()` question. We usually do **not** care about the total of all cylinder values added together. That would be a `sum()` question, and it would not be very meaningful here.

A frequency table helps describe categorical or discrete data. The mode identifies the value that appears most often.

Add this heading to your Quarto file:

```
# Count-Based Summaries
```

6.1 Task 6

Choose count-based summary variables

Under `# Count-Based Summaries`, decide which variables in the `penguins` dataset are appropriate for `length()`, `count()`, and mode reporting.

Goal: Identify variables that make sense for count-based summaries and begin writing the Count-Based Summaries section of the Data Understanding Notebook.

6.1.1 Step 1

Review the columns in `penguins`.

Choose:

1. one column you could use with `length()` to report how many values are in the dataset
2. two columns that make sense to summarize with `count()`
3. one column where the mode would be meaningful

Tip: `count()` and mode are most useful for categorical, factor, and discrete variables.

6.1.2 Step 2

Create one inline R sentence using `length()`.

Copy the sentence pattern below into your report. Replace each `_____` with your chosen column name.

```
**Comment:** The `_____` column in the penguins dataset contains {r} length(penguins$_____) observ
```

Then add one sentence explaining your choice.

```
<!-- Example in Quarto -->
**Comment:** I chose `_____` for `length()` because it is a _____ variable. It makes sense to use `le
```

6.1.3 Step 3

Create one frequency table using `count()`.

Replace _____ with one appropriate column name.

```
#
penguin_counts <- penguins %>%
  count(_____)
```

Print the table so it appears in your HTML report.

```
#
penguin_counts
```

6.1.4 Step 4

Identify the mode from your frequency table.

Look at the frequency table and find the value with the largest `n`. That value is the mode.

Then write 2–3 sentences explaining:

- which variable you chose for `count()`
- why that variable is appropriate for a frequency table
- what the mode tells you about the dataset

Use this sentence pattern if you need help getting started:

```
**Comment:** I chose `_____` for `count()` because it is a _____ variable. It makes sense to use `cou

**Comment:** The most common value in the `_____` column is _____. This means _____ appears most often
```

6.1.5 Step 5

Render your Quarto document to HTML.

Checks: Make sure the `# Count-Based Summaries` section includes:

- successful runs of inline R sentences
- one frequency table using `count()`
- at least two complete comment sentences
- one statement identifying the mode

Reflection: What is one variable in `penguins` that would **not** make sense to summarize with `count()` and mode? Explain why.

The next sections introduce two other measures of center that are more appropriate for numerical data.

Measures of **central tendency**, or measures of center, are used to describe the typical value in a dataset. They give us an idea of where the center of the data is located. Common measures of central tendency include the mean, median, and mode.

You have already used mode, a measure of center because it identifies the value that appears most often. The next section introduces two other measures of center: **median** and **mean**. These are more appropriate for numerical data.

This section also adds a new Quarto formatting skill: writing mathematical notation in the report. You will still practice inline R code, but you will also see how math text is written with dollar signs \$ and \$\$ for inline and block math, respectively.

7 Measures of Center for Numerical Data

Pause report-building for a moment as you are entering another reading section.

This reading explains how the **median** and **mean** describe the center of numerical data. It also shows how to write math notation in a Quarto report.

After the reading, you will return to your Data Understanding Notebook Quarto file and add a new report section called **# Measures of Center for Numerical Data**.

7.1 Median

The **median** is the middle value of a set of numbers when the values are arranged in order. The median is often useful when the data may contain unusually high or low values.

To find the median:

Step 1: Sort the data in ascending order.

Step 2: Check the size of the sample, n .

Step 3: Calculate the median using the formula for an odd or even number of values.

If n is odd, the median is the middle value.

```
<!-- Example in Quarto -->
$$
\text{median} = x_{\{\frac{n+1}{2}\}}
$$
```

This is what it looks like after rendering:

$$\text{median} = x_{(\frac{n+1}{2})}$$

Notice: In $\frac{n+1}{2}$, the first set of braces, $\{n+1\}$, becomes the numerator, or top of the fraction. The second set of braces, $\{2\}$, becomes the denominator, or bottom of the fraction.

If n is even, the median is the average of the two middle values.

```
<!-- Example in Quarto -->
$$
\text{median} = \frac{x_{\{\frac{n}{2}\}} + x_{\{\frac{n}{2}+1\}}}{2}
$$
```

This is what it looks like after rendering:

$$\text{median} = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$$

Notice: In this example, $\frac{\{ \}}{\{ \}}$ appears more than once. The larger fraction has $x_{\{\frac{n}{2}\}} + x_{\{\frac{n}{2}+1\}}$ in the first set of braces, so that expression becomes the numerator. The second set of braces, $\{2\}$, becomes the denominator. Inside the numerator, smaller fractions such as $\frac{n}{2}$ follow the same pattern.

Where $x_{(i)}$ represents the i th ordered value in the sample.

Goal: Calculate the median of the `mpg` column in the `mtcars` dataset.

First, check the values.

```
#
print(mtcars$mpg)

## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

Sort the values in ascending order.

```
#
sorted_data <- sort(mtcars$mpg)

#
sorted_data

## [1] 10.4 10.4 13.3 14.3 14.7 15.0 15.2 15.2 15.5 15.8 16.4 17.3 17.8 18.1 18.7
## [16] 19.2 19.2 19.7 21.0 21.0 21.4 21.4 21.5 22.8 22.8 24.4 26.0 27.3 30.4 30.4
## [31] 32.4 33.9

<!-- Example in Quarto -->
**Comment:** The `mpg` column in the `mtcars` dataset has been sorted in ascending order.
```

This is what it looks like after rendering:

Comment: The mpg column in the mtcars dataset has been sorted in ascending order.

Find the size of the sample.

```
#
length(sorted_data)

## [1] 32

<!-- Example in Quarto -->
**Comment:** There are {r} length(sorted_data) values.
```

This is what it looks like after rendering:

Comment: There are 32 values.

Because there are 32 values, we need the median for a dataset with an even number of data points. The two middle values are the 16th and 17th values in the sorted list.

Use bracket indexing to access the 16th and 17th values, then take their average.

```
#
median_value <- (sorted_data[16] + sorted_data[17]) / 2

#
median_value
```

```
## [1] 19.2
```

R also has a median() function that does this calculation directly.

```
#
median(mtcars$mpg)

## [1] 19.2
```

```
<!-- Example in Quarto -->
**Comment:** The median of the `mpg` column in the `mtcars` dataset is  $\{r\}$  median(mtcars$mpg).
```

This is what it looks like after rendering:

Comment: The median of the mpg column in the mtcars dataset is 19.2.

The median is a measure of center that is often selected when the data contain unusually high or low values. The next section introduces the mean, which is the average of the values in a dataset.

8 Practice Space 3

Now return to your Quarto file.

In this Practice Space, you will add to the report section called `# Measures of Center for Numerical Data`.

8.1 Task 7

Report the median

Place this task after the **Median** section in your Data Understanding Notebook.

Goal: Choose one numerical measurement from the `penguins` dataset and explain why the median is an appropriate measure of center for that variable.

8.1.1 Step 1

Choose one numerical measurement variable from `penguins`.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

8.1.2 Step 2

Add a short math-text statement before your code.

Math text is useful in data notebooks because it helps readers understand the method before they see the R calculation. Data analysts often include a formula or short math statement before code when they want the report to explain both the idea and the calculation.

Copy and paste this into your report:

The median is the middle value after the data are sorted. For an odd number of values, the median position is

```
$$
\text{median position} = \frac{n+1}{2}
$$
```

8.1.3 Step 3

Calculate the median for your chosen variable.

Replace `_____` with the column name you chose.


```
#
median(penguins$_____, na.rm = TRUE)
```

Tip: If your variable contains missing values, include `na.rm = TRUE`.

8.1.4 Step 4

Write an inline R sentence that reports the median value.

Replace each `_____` with the column name you chose.

```
**Comment:** The median of the `_____` column in the `penguins` dataset is {r} median(penguins$_____,
```

Render to HTML to see the result.

Checks: Make sure the sentence shows a number, not the inline R code.

8.1.5 Step 5

Write a short reflection.

Use the sentence pattern below to explain why the median makes sense for your chosen variable.

```
**Reflection:** I chose the `_____` variable from the `penguins` dataset to describe with the median b
```

The median is useful when data may contain unusually high or low values. The next section introduces the **mean**, another measure of center that is commonly used for numerical data.

Pause report-building while you review the Mean reading section.

8.2 Mean

The **mean** is the average of a set of numbers. It is calculated by summing all the values and dividing by the number of values.

Step 1a is to count the number of values in the dataset.

Step 1b is to sum the values in the dataset.

Step 2 is to divide the sum by the number of values in the dataset.

8.2.1 Step 1a

We calculated the number of values in the `mpg` column of the `mtcars` dataset in the previous section.

```
# length (Step 1a)
n <- length(mtcars$mpg)
```

8.2.2 Step 1b

Summation is the process of adding a sequence of numbers together. Summation is represented by the Greek letter sigma (Σ). It indicates that we are summing a series of values.

```
<!-- Example in Quarto -->
$$
\sum_{i=1}^n
$$
```

This is what it looks like after rendering:

$$\sum_{i=1}^n$$

Notice: In `\sum_{i=1}^n`, the `_i` places text below the summation symbol, and the `^n` places text above it. The `\sum` creates the summation symbol.

When you see $\sum_{i=1}^5 x_i$, this means:

- Start with $i = 1$ (the first value in the set of numbers)
- Add the value x_1 to your sum
- Increase i by 1
- Repeat until you reach $i = 5$ (the fifth value in the set of numbers)
- Then stop

This is adding five values: x_1, x_2, x_3, x_4 , and x_5 .

```
<!-- Example in Quarto -->
$$
\sum_{i=1}^5 x_i = x_1 + x_2 + x_3 + x_4 + x_5
$$
```

This is what it looks like after rendering:

$$\sum_{i=1}^5 x_i = x_1 + x_2 + x_3 + x_4 + x_5$$

Notice: In `x_i`, the `_i` creates a subscript. The same pattern is used in `x_1, x_2, x_3, x_4`, and `x_5`.

Sum the values in the `mpg` column of the `mtcars` dataset. Store the result as `sum_x`.

```
# Sum (Step 1b)
sum_x <- sum(mtcars$mpg)

#
sum_x
```

```
## [1] 642.9
```

`sum(mtcars$mpg)` is the total value from adding all the values in the `mpg` column of the `mtcars` dataset.

```
<!-- Example in Quarto -->
**Comment:** The sum of the values in the `mpg` column of the `mtcars` dataset is {r} sum_x.
```

This is what it looks like after rendering:

Comment: The sum of the values in the `mpg` column of the `mtcars` dataset is 642.9.

You can also use `sum(mtcars$mpg)` directly in the inline code without storing it in a variable.

```
<!-- Example in Quarto -->
**Comment:** The sum of the values in the `mpg` column of the `mtcars` dataset is {r} sum(mtcars$mpg)
```

This is what it looks like after rendering:

Comment: The sum of the values in the `mpg` column of the `mtcars` dataset is 642.9.

8.2.3 Step 2

Divide the sum of the values by the number of values to get the mean.

The mean calculation can be written mathematically using Quarto math text.

```
<!-- Example in Quarto -->
$$
\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i
$$
```

This is what it looks like after rendering:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Notice: In `\bar{x}`, the `\bar{}` adds a bar over `x`. In `\frac{1}{n}`, the first set of braces, `{1}`, becomes the numerator, and the second set of braces, `{n}`, becomes the denominator. In `\sum_{i=1}^n`, the `_` places text below the summation symbol, and the `^` places text above it.

Calculate the mean of the `mpg` column in the `mtcars` dataset.

```
# Mean (Step 2) Divide Step 1 values
x_bar <- sum_x / n

#
x_bar
```

```
## [1] 20.09062
```

Comment: We used the `sum_x` and `n` objects created earlier.

Calculate the mean of the `mpg` column in the `mtcars` dataset using the `mean()` function.

```
#
# OR use mean() function directly
x_bar <- mean(mtcars$mpg)

#
x_bar
```

```
## [1] 20.09062
```

```
<!-- Example in Quarto -->
**Comment:** The mean of the values in the `mpg` column of the `mtcars` dataset is {r} mean(mtcars$mpg)
```

This is what it looks like after rendering:

Comment: The mean of the values in the `mpg` column of the `mtcars` dataset is 20.090625.

9 Practice Space 4

Now return to your Quarto file.

In this Practice Space, you will continue building the report section called `# Measures of Center for Numerical Data`.

You will calculate the mean, compare it to the median, and then compare a measure of center across groups. This gives you more practice with inline R code while helping you decide which measure of center is useful for different numerical variables.

You are not expected to write math notation from memory. In this section, your goal is to recognize what the math text is showing and use R to calculate and report the values.

9.1 Task 8

Compare mean and median

Goal: Calculate and report the mean for one numerical measurement from the `penguins` dataset, then compare it to the median.

9.1.1 Step 1

Choose one numerical measurement variable.

You may use the same variable you used for median. That is the best choice if you want to compare mean and median clearly.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

9.1.2 Step 2

Calculate the mean for your chosen variable.

Replace `_____` with the column name you chose.

```
#  
mean(penguins$_____, na.rm = TRUE)
```

Tip: If your variable contains missing values, include `na.rm = TRUE`.

9.1.3 Step 3

Write one inline R sentence that reports both the median and the mean.

Replace each `_____` with the column name you chose.

```
**Comment:** For the `_____` column, the median is {r} median(penguins$_____, na.rm = TRUE) and the mean is {r} mean(penguins$_____, na.rm = TRUE)
```

Render to HTML to see the result.

Checks: Make sure the sentence shows numbers, not the inline R code.

9.1.4 Step 4

Reflection: Write 2–3 sentences comparing the mean and median.

Use this sentence pattern if you need help getting started:

```
**Reflection:** I chose the `_____` variable because it is a numerical measurement variable. The mean is {r} mean(penguins$_____, na.rm = TRUE) and the median is {r} median(penguins$_____, na.rm = TRUE)
```

9.2 Task 9

Compare center across groups

Place this task under a new heading called `## Measures of Center Compared`.

This task moves from describing one numerical variable to comparing a measure of center across groups.

Goal: Compare the mean or median for one numerical variable across two groups from one categorical variable.

9.2.1 Step 1

Add this heading to your Quarto file:

```
## Measures of Center Compared
```

9.2.2 Step 2

Choose one numerical measurement variable and one categorical grouping variable.

Choose one numerical variable:

- bill_length_mm
- bill_depth_mm
- flipper_length_mm
- body_mass_g

Choose one grouping variable:

- species
- island
- sex

For example, you might compare body mass by sex or bill length by species.

9.2.3 Step 3

Calculate the mean or median for two groups.

Use one of the example structures below. Replace each _____ with your chosen variable, grouping variable, and group names.

Example using median:

```
#
median(penguins$_____ [penguins$_____ == "_____"], na.rm = TRUE)
median(penguins$_____ [penguins$_____ == "_____"], na.rm = TRUE)
```

Example using mean:

```
#
mean(penguins$_____ [penguins$_____ == "_____"], na.rm = TRUE)
mean(penguins$_____ [penguins$_____ == "_____"], na.rm = TRUE)
```

Tip: Use `unique(penguins$_____)` to check the exact group names before typing them into your code.

9.2.4 Step 4

Write one inline R sentence that reports both group values.

Use the median version or the mean version.

```
**Comment:** The median `_____` for `_____` penguins is  $\{r\}$  median(penguins$_____ [penguins$_____ == "_____"])
**Comment:** The mean `_____` for `_____` penguins is  $\{r\}$  mean(penguins$_____ [penguins$_____ == "_____"])
```

Render to HTML to see the result.

Checks: Make sure the sentence shows numbers, not the inline R code.

9.2.5 Step 5

Reflection: Write 2–3 sentences interpreting the comparison.

Use this sentence pattern if you need help getting started:

```
**Reflection:** I compared `_____` across two groups from the `_____` variable. The group with the larger
```

This comparison helps you see an early pattern in the data and suggests a direction for later descriptive analysis.

Measures of center tell us about a typical value. Measures of spread tell us how tightly clustered or spread out the values are around that center.

10 Measures of Spread

Measures of **spread** describe how much the values in a dataset vary. In this section, you will review standard deviation, range, interquartile range, and the five-number summary. These measures help analysts understand whether data values are relatively consistent or whether they vary widely.

This section also introduces two new **Quarto formatting** skills that help an HTML report read like an analyst’s notebook rather than a list of answers:

- callout notes
- section links

Pause report-building while you review the Measures of Spread reading section.

10.1 Standard Deviation

The **standard deviation** is a measure of how spread out the values in a dataset are. It is calculated by taking the square root of the variance.

The **variance** is based on how far each value is from the mean. To calculate the sample variance:

1. Find the difference between each value and the mean.
2. Square those differences.
3. Add the squared differences together.
4. Divide by the number of values minus 1.

This results in the sample variance, which is written as s^2 . Then we take the square root of the variance to get the standard deviation.

10.1.1 Variance Formula

The variance is based on how far each value is from the mean. We can build the formula one piece at a time.

1. First, each value is compared to the mean.

```
<!-- Example in Quarto -->
$$
x_i - \bar{x}
$$
```

This is what it looks like after rendering:

$$x_i - \bar{x}$$

This expression means: one data value minus the mean.

Notice: In `x_i`, the `_i` creates a subscript. In `\bar{x}`, the `\bar{}` adds a bar over `x`. $\bar{x}$ is the mean value.

2. Next, each difference is squared.

```
<!-- Example in Quarto -->
$$
(x_i - \bar{x})^2
$$
```

This is what it looks like after rendering:

$$(x_i - \bar{x})^2$$

This expression means: square the distance between one data value and the mean.

Notice: The parentheses in `(x_i - \bar{x})^2` show that the whole difference is squared. The `^2` creates the superscript 2.

3. Next, all of the squared differences are added together.

```
<!-- Example in Quarto -->
$$
\sum_{i=1}^n (x_i - \bar{x})^2
$$
```

This is what it looks like after rendering:

$$\sum_{i=1}^n (x_i - \bar{x})^2$$

This expression means: add the squared differences for all values in the dataset.

Notice: In `\sum_{i=1}^n`, the `\sum` creates the summation symbol. The `_{i=1}` places `i=1` below the symbol, and the `^n` places `n` above the symbol.

4. Finally, divide the sum of squared differences by the number of values minus 1.

```
<!-- Example in Quarto -->
$$
s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}
$$
```

This is what it looks like after rendering:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

This full formula represents the sample variance. The s^2 represents the sample variance, and $n - 1$ is the denominator used for a sample variance calculation.

Notice: In `s^2`, the `^2` creates a superscript. In `\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}`, the first set of braces becomes the numerator, or top of the fraction. The second set of braces becomes the denominator, or bottom of the fraction.

The R code below follows the same structure as the formula. It subtracts the mean from each value, squares those differences, adds them together, and divides by `n - 1`.

Goal: Calculate the variance of the `mpg` column in the `mtcars` dataset.

```
#
x_bar <- mean(mtcars$mpg)
n <- length(mtcars$mpg)

variance <- sum((mtcars$mpg - x_bar)^2) / (n - 1)

#
variance

## [1] 36.3241
```

This calculation uses objects in steps:

- `x_bar` stores the mean of the `mpg` column.
- `n` stores the number of values in the `mpg` column.
- `mtcars$mpg - x_bar` calculates the difference between each value and the mean.
- `(mtcars$mpg - x_bar)^2` squares each difference.
- `sum((mtcars$mpg - x_bar)^2)` adds the squared differences together.
- `/ (n - 1)` divides by the number of values minus 1.

This results in the sample variance, or s^2 .

R also has a built-in function for variance.

Goal: Calculate the variance of the `mpg` column using `var()`.

```
#
variance <- var(mtcars$mpg)

#
variance

## [1] 36.3241
```

A Quarto report can use inline R code to place the variance inside a sentence.

```
<!-- Example in Quarto -->
**Comment:** The variance of the values in the `mpg` column of the `mtcars` dataset is {r} round(var(
```

This is what it looks like after rendering:

Comment: The variance of the values in the `mpg` column of the `mtcars` dataset is 36.32.

10.1.2 Standard Deviation Formula

The standard deviation is found by taking the square root of the variance.

The formula can be written in a Quarto report using block math.

```
<!-- Example in Quarto -->
$$
s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}
$$
```

This is what it looks like after rendering:

$$s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

The s represents the sample standard deviation. The s^2 represents the sample variance. Taking the square root changes the variance back into the original unit of measurement.

Notice: In `\sqrt{s^2}`, the `\sqrt{}` creates a square root symbol around the content inside the braces.

Notice: In this example, the fraction is placed inside `\sqrt{}`. That means the entire fraction appears under the square root symbol after rendering.

Goal: Calculate the standard deviation of the `mpg` column in the `mtcars` dataset.

```
#
std_dev <- sqrt(sum((mtcars$mpg - x_bar)^2) / (n - 1))

#
std_dev
```

```
## [1] 6.026948
```

This places the variance calculation inside `sqrt()`, which calculates the square root of the variance.

R also has a built-in function for standard deviation.

Goal: Calculate the standard deviation of the `mpg` column using `sd()`.

```
#
std_dev <- sd(mtcars$mpg)

#
std_dev
```

```
## [1] 6.026948
```

A Quarto report can also use inline R code to place the standard deviation inside a sentence.

```
<!-- Example in Quarto -->
**Comment:** The standard deviation of the values in the `mpg` column of the `mtcars` dataset is {r}.
```

This is what it looks like after rendering:

Comment: The standard deviation of the values in the `mpg` column of the `mtcars` dataset is 6.03.

11 Practice Space 5

Now return to your Quarto file.

In this Practice Space, you will continue building the report section called **# Measures of Spread**.

You will add a standard deviation subsection, label sections so Quarto can link to them, calculate a standard deviation from the `penguins` dataset, and use a callout note to make an interpretation stand out.

Tip: Render to HTML after major edits. Since this activity teaches report formatting, you need to check both your R output and how the report looks after rendering. Correct any errors in R code or formatting before moving on to the next task.

11.1 Task 10

Update YAML and add section labels

Goal: Turn on numbered sections and add section labels so Quarto can create section links later in your report.

11.1.1 Step 1

Update the YAML at the top of your Quarto file.

Your YAML should look like this:

```
---  
title: "Your Title"  
format: html  
number-sections: true  
---
```

The line `number-sections: true` tells Quarto to number the headings in your HTML report. This makes section links easier for readers to follow.

11.1.2 Step 2

Add labels to the section headings in your Measures of Spread section.

A **section label** is a short name attached to a heading. Quarto can use that label to create a link to that section later in your report.

Copy and paste these headings into your report as you build each subsection:

```
## Standard Deviation {#sec-sd}  
  
## Range {#sec-range}  
  
## Interquartile Range {#sec-iqr}  
  
## Five-Number Summary {#sec-five}
```

Each label goes inside curly braces after the heading text. The label starts with `#` and uses a short unique name.

For example:

- `{#sec-sd}` labels the Standard Deviation section.
- `{#sec-range}` labels the Range section.
- `{#sec-iqr}` labels the Interquartile Range section.
- `{#sec-five}` labels the Five-Number Summary section.

11.1.3 Step 3

Add a label to your earlier Measures of Center section.

Find this heading in your report:

```
# Measures of Center for Numerical Data
```

Change it to:

```
# Measures of Center for Numerical Data {#sec-center}
```

You will use `@sec-center` later to create a link back to that section.

11.1.4 Step 4

Checks: Render to HTML.

After rendering, check that:

- your section headings are numbered

- your Measures of Spread headings still appear correctly
- the labels such as `{#sec-sd}` and `{#sec-center}` do **not** show as regular text in the final HTML report

If the labels show as regular text, check that you typed them at the end of the heading line with curly braces.

11.2 Task 11

Add the standard deviation formula

Goal: Add a short math-text statement to the Standard Deviation section of your report.

11.2.1 Step 1

Go to this heading in your Quarto file:

```
## Standard Deviation {#sec-sd}
```

11.2.2 Step 2

Copy and paste this into your report:

The standard deviation is the square root of the variance.

```
$$
s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}
$$
```

11.2.3 Step 3

Checks: Render to HTML.

After rendering, check that:

- the formula appears centered on its own line
- the square root symbol appears correctly
- the fraction appears inside the square root
- the raw `$$` symbols do **not** show in the rendered report

11.3 Task 12

Report the standard deviation

Goal: Calculate and report the standard deviation for one numerical measurement from the `penguins` dataset.

11.3.1 Step 1

Choose one numerical measurement variable from the `penguins` dataset.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

Choose a measurement variable, not a categorical variable like `species`, `island`, or `sex`.

11.3.2 Step 2

Calculate the standard deviation for your chosen variable.

Replace `_____` with the column name you chose.

```
#  
sd(penguins$_____, na.rm = TRUE)
```

11.3.3 Step 3

Add a sentence that reports the standard deviation using inline R code.

Replace each `_____` with the column name you chose.

```
**Comment:** The standard deviation of the `_____` column in the `penguins` dataset is {r} round(sd(p
```

Checks: Render to HTML.

After rendering, check that:

- your sentence shows a number, not raw inline R code
- the number is rounded to two decimal places
- the variable name matches the column you chose

11.3.4 Step 4

Add a callout note with your interpretation.

A **callout note** creates a shaded box in your HTML report. It is useful when you want an important interpretation, warning, reminder, or takeaway to stand out from the rest of the text.

In Quarto, a callout note starts with this line:

```
::: {.callout-note}
```

The callout note ends with this line:

```
:::
```

Everything between those two lines appears inside the shaded note box after rendering.

Copy and paste this into your report. Replace `_____` with the column name you chose.

```
::: {.callout-note}
```

```
**Reflection:** I chose the `_____` variable because it is a numerical measurement. The standard dev
```

```
:::
```

Checks: Render to HTML.

After rendering, check that:

- the reflection appears inside a shaded callout box
- the callout markers `::: {.callout-note}` and `:::` do **not** show as regular text
- your chosen variable name appears in the reflection

11.3.5 Step 5

Add one sentence that links back to your Measures of Center section.

A **section link** points the reader to another section of your report. Since you labeled the Measures of Center section as `{#sec-center}`, you can link to it by writing `@sec-center`.

Copy and paste this sentence below your standard deviation callout note:

```
**Comment:** Compared with the center measures discussed in @sec-center, the standard deviation helps c
```

Checks: Render to HTML.

After rendering, check that:

- `@sec-center` turns into a clickable section link
- the link points back to the Measures of Center section
- the raw text `@sec-center` does **not** remain unchanged in the final HTML report

Standard deviation uses every value in the dataset, so it gives a fuller picture of variability than a measure based on only the smallest and largest values. However, standard deviation is affected by unusually high or low values. This means it can be less helpful when a distribution is strongly skewed or contains outliers.

In the next section, we will explore measures of spread that are less sensitive to outliers and skewed data.

Pause report-building while you review the Range reading section.

11.4 Range

The simplest measure of spread to calculate and understand is the **range**. The range is the difference between the largest and smallest values in a dataset.

The range gives you a quick sense of how spread out the data is, but it can be heavily influenced by outliers because it depends only on two values.

The range formula can be written in a Quarto report using block math.

```
<!-- Example in Quarto -->
$$
\text{Range} = x_{(n)} - x_{(1)}
$$
```

This is what it looks like after rendering:

$$\text{Range} = x_{(n)} - x_{(1)}$$

In this formula, $x_{(n)}$ represents the largest value and $x_{(1)}$ represents the smallest value.

Notice: In $x_{(n)}$ and $x_{(1)}$, the $_{\{}$ creates a subscript. The parentheses are included inside the subscript to show that the values are ordered from smallest to largest.

In R, you can use the `range()` function to return both the minimum and maximum values.

Goal: Find the minimum and maximum values of the `mpg` column in the `mtcars` dataset.

```
#
mpg_range <- range(mtcars$mpg)

#
mpg_range

## [1] 10.4 33.9
```

The object `mpg_range` is a vector that contains two values from the `mpg` column:

- the first value is the minimum
- the second value is the maximum

We can use the bracket operator `[]` to access each value from the `mpg_range` vector.

Goal: Calculate the range by subtracting the minimum from the maximum.

```
#
mpg_range_value <- mpg_range[2] - mpg_range[1]

#
mpg_range_value

## [1] 23.5
```

You can also calculate the range directly by using the `max()` and `min()` functions.

Goal: Calculate the range directly.

```
#
max(mtcars$mpg) - min(mtcars$mpg)

## [1] 23.5
```

Because range depends only on the minimum and maximum values, one extreme value can make it misleading. It also does not show where most of the data fall. The next section introduces a measure of spread that is less affected by extreme values.

12 Practice Space 6

Now return to your Quarto file.

In this Practice Space, you will continue building the report section called **# Measures of Spread**.

You will add to the subsection called **## Range {#sec-range}**. You will calculate the range for one numerical measurement from the `penguins` dataset, report the minimum, maximum, and range in a Markdown list, and write a short interpretation.

Render to HTML after major edits so you can check that your list, inline R code, callout note, and section link display correctly.

12.1 Task 13

Report the range

Goal: Calculate and report the range for one numerical measurement from the `penguins` dataset.

12.1.1 Step 1

Choose one numerical measurement variable from the `penguins` dataset.

You may use the same numerical variable you used for standard deviation, or you may choose another numerical measurement variable.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

Choose a measurement variable, not a categorical variable like `species`, `island`, or `sex`.

12.1.2 Step 2

Calculate the minimum and maximum values for your chosen variable.

Replace `_____` with the column name you chose.

```
#
penguin_range <- range(penguins$_____, na.rm = TRUE)

#
penguin_range
```

12.1.3 Step 3

Calculate the range value.

The `range()` function gives you two values. The first value is the minimum, and the second value is the maximum.

To calculate the range, subtract the first value from the second value.

```
#
penguin_range_value <- penguin_range[2] - penguin_range[1]

#
penguin_range_value
```

12.1.4 Step 4

Report your results as a short Markdown list.

A Markdown list helps readers scan related results quickly. In this list, you will report the minimum, maximum, and range.

Replace each `_____` with the column name you chose.

```
- **Minimum**: `{r} range(penguins$_____, na.rm = TRUE)[1]`
- **Maximum**: `{r} range(penguins$_____, na.rm = TRUE)[2]`
- **Range**: `{r} diff(range(penguins$_____, na.rm = TRUE))`
```

Checks: Render to HTML.

After rendering, check that:

- the results appear as a bulleted list
- each list item shows a number, not raw inline R code
- the minimum, maximum, and range values make sense for the variable you chose

12.1.5 Step 5

Add a callout note that compares range to measures of center.

A section link helps readers move between related parts of your report. Since you labeled the Measures of Center section as `{#sec-center}`, you can link to it by writing `@sec-center`.

Copy and paste this into your report:

```
::: {.callout-note}

  **Comment:** Compared with the center measures discussed in @sec-center, the range gives a quick descr

:::
```

Checks: Render to HTML.

After rendering, check that:

- the sentence appears inside a shaded callout box
- `@sec-center` turns into a clickable section link
- the link points back to the Measures of Center section
- the raw text `@sec-center` does **not** remain unchanged in the final HTML report

If the link does not work, check that your earlier heading looks like this:

```
# Measures of Center for Numerical Data {#sec-center}
```

12.1.6 Step 6

Add a short reflection about the range.

The range is easy to calculate, but it only uses two values: the minimum and maximum. Your reflection should explain both why the range is useful and why it can be limited.

Copy and paste this sentence pattern into your report. You may edit it to better match your variable.

```
**Reflection:** The range gives a quick description of how far apart the smallest and largest values a
```

Range gives a quick description of spread because it compares the largest and smallest values. This makes it easy to understand, but it also means the range can be strongly affected by one unusually high or low value.

The interquartile range, or IQR, gives a different view of spread. Instead of using the full distance from the smallest value to the largest value, IQR focuses on the middle 50% of the data.

Pause report-building while you review the Interquartile Range reading section.

12.2 Interquartile Range (IQR)

The **interquartile range** (IQR) describes the spread of the middle 50% of the data. It is often useful because it is less affected by unusually high or low values than the full range.

A **quartile** divides ordered data into four parts.

- Q_1 is the first quartile, or 25th percentile.
- Q_2 is the second quartile, or median.
- Q_3 is the third quartile, or 75th percentile.

The IQR is calculated by subtracting the first quartile from the third quartile.

The formula can be written in a Quarto report using block math.

```
<!-- Example in Quarto -->
$$
IQR = Q_3 - Q_1
$$
```

This is what it looks like after rendering:

$$IQR = Q_3 - Q_1$$

In this formula, Q_1 represents the first quartile and Q_3 represents the third quartile.

Notice: In `Q_1` and `Q_3`, the `_1` and `_3` create subscripts.

By focusing on the middle 50% of the data, the IQR reduces the influence of extreme values.

Goal: Calculate the interquartile range of the `mpg` column in the `mtcars` dataset.

```
#  
iqr_value <- IQR(mtcars$mpg)  
  
#  
iqr_value  
  
## [1] 7.375
```

A Quarto report can use inline R code to place the IQR inside a sentence.

```
<!-- Example in Quarto -->  
**Comment:** The interquartile range of the values in the `mpg` column of the `mtcars` dataset is `{r}
```

This is what it looks like after rendering:

Comment: The interquartile range of the values in the `mpg` column of the `mtcars` dataset is 7.38.

The IQR gives a narrower view of spread than the full range because it ignores the lowest 25% and highest 25% of values. This makes it useful when a dataset has skew or possible outliers.

13 Practice Space

Now return to your Quarto file.

In this Practice Space, you will continue building the report section called `# Measures of Spread`.

You will add to the subsection called `## Interquartile Range {#sec-iqr}`. You will calculate the IQR for one numerical measurement from the `penguins` dataset, report the result with inline R code, write a short reflection, and add a section link back to your Range section.

Render to HTML after major edits so you can check that your inline R code and section link display correctly.

13.1 Task 14

Report the interquartile range

Goal: Calculate and report the interquartile range (IQR) for one numerical measurement from the `penguins` dataset.

13.1.1 Step 1

Choose one numerical measurement variable from the `penguins` dataset.

You may use the same numerical variable you used for standard deviation or range, or you may choose another numerical measurement variable.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

Choose a measurement variable, not a categorical variable like `species`, `island`, or `sex`.

13.1.2 Step 2

Calculate the interquartile range for your chosen variable.

Replace `_____` with the column name you chose.

```
#  
iqr_value <- IQR(penguins$_____, na.rm = TRUE)  
  
#  
iqr_value
```

13.1.3 Step 3

Add a sentence that reports the interquartile range using inline R code.

Replace each `_____` with the column name you chose.

```
**Comment:** The interquartile range of the _____ column in the penguins dataset is {r} round(IQR
```

Checks: Render to HTML.

After rendering, check that:

- your sentence shows a number, not raw inline R code
- the number is rounded to two decimal places
- the variable name matches the column you chose

13.1.4 Step 4

Write a short reflection about the IQR.

The IQR describes the spread of the middle 50% of the data. Your reflection should explain why that is useful when a dataset may have unusually high or low values.

Copy and paste this sentence pattern into your report. You may edit it to better match your variable.

```
**Reflection:** The IQR describes the spread of the middle 50% of the data. It is useful because it is
```

13.1.5 Step 5

Add one comparison sentence that links back to the Range section.

A section link helps readers connect related ideas in your report. Since you labeled the Range section as `{#sec-range}`, you can link to it by writing `@sec-range`.

Copy and paste this sentence below your IQR reflection:

```
**Comment:** Compared with the full spread described in @sec-range, the IQR focuses on the middle 50% of
```

Checks: Render to HTML.

After rendering, check that:

- `@sec-range` turns into a clickable section link
- the link points back to the Range section
- the raw text `@sec-range` does **not** remain unchanged in the final HTML report

If the link does not work, check that your Range heading looks like this:

```
## Range {#sec-range}
```

The IQR focuses on the middle 50% of the data by comparing Q_3 and Q_1 . The five-number summary expands that idea by showing the low end, the quartiles, the middle, and the high end of a numeric variable.

Pause report-building while you review the Five-Number Summary reading section.

13.2 Five-Number Summary

A **five-number summary** gives a quick overview of the spread and center of a numeric variable.

The five values are:

- minimum
- Q_1 , or first quartile
- median
- Q_3 , or third quartile
- maximum

These values help you see the low end, the middle 50%, the median, and the high end of the data.

The five-number summary can be written in a Quarto report using a short Markdown list.

```
<!-- Example in Quarto -->
- **Minimum**
- **Q1**
- **Median**
- **Q3**
- **Maximum**
```

This is what it looks like after rendering:

- **Minimum**
- **Q1**
- **Median**
- **Q3**
- **Maximum**

Notice: The - creates a bullet point in Markdown. The double asterisks in **Minimum** make the word bold after rendering.

In R, the `summary()` function gives a quick overview of a numeric variable. For a numeric variable, `summary()` returns the minimum, Q_1 , median, mean, Q_3 , and maximum.

That means `summary()` shows the five-number summary **plus** the mean.

Goal: Use `summary()` to get a quick overview of all columns in the `mtcars` dataset.

```
#
summary(mtcars)
```

##	mpg	cyl	disp	hp
##	Min. :10.40	Min. :4.000	Min. : 71.1	Min. : 52.0
##	1st Qu.:15.43	1st Qu.:4.000	1st Qu.:120.8	1st Qu.: 96.5
##	Median :19.20	Median :6.000	Median :196.3	Median :123.0
##	Mean :20.09	Mean :6.188	Mean :230.7	Mean :146.7
##	3rd Qu.:22.80	3rd Qu.:8.000	3rd Qu.:326.0	3rd Qu.:180.0
##	Max. :33.90	Max. :8.000	Max. :472.0	Max. :335.0
##	drat	wt	qsec	vs
##	Min. :2.760	Min. :1.513	Min. :14.50	Min. :0.0000
##	1st Qu.:3.080	1st Qu.:2.581	1st Qu.:16.89	1st Qu.:0.0000
##	Median :3.695	Median :3.325	Median :17.71	Median :0.0000
##	Mean :3.597	Mean :3.217	Mean :17.85	Mean :0.4375

```
## 3rd Qu.:3.920 3rd Qu.:3.610 3rd Qu.:18.90 3rd Qu.:1.0000
## Max. :4.930 Max. :5.424 Max. :22.90 Max. :1.0000
##      am      gear      carb
## Min. :0.0000 Min. :3.000 Min. :1.000
## 1st Qu.:0.0000 1st Qu.:3.000 1st Qu.:2.000
## Median :0.0000 Median :4.000 Median :2.000
## Mean   :0.4062 Mean   :3.688 Mean   :2.812
## 3rd Qu.:1.0000 3rd Qu.:4.000 3rd Qu.:4.000
## Max.   :1.0000 Max.   :5.000 Max.   :8.000
```

This summarizes all columns at once. That can be useful for a quick scan, but it may include more output than you need.

Goal: Use `summary()` to get a quick overview of only the `mpg` column.

```
#
summary(mtcars$mpg)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.40  15.43   19.20   20.09  22.80   33.90
```

This output is easier to read because it focuses on one numeric variable.

The five-number summary is useful because it connects several measures of spread. The minimum and maximum show the full range, while Q_1 and Q_3 help describe the middle 50% of the data. The median shows the center of the ordered values.

14 Practice Space

Now return to your Quarto file.

In this Practice Space, you will finish building the report section called `# Measures of Spread`.

You will add to the subsection called `## Five-Number Summary {#sec-five}`. You will use `summary()` to describe one numerical measurement from the `penguins` dataset, write a short interpretation, and end the Measures of Spread section with a takeaway.

Render to HTML after major edits so you can check that your code output, report text, and final takeaway display correctly.

14.1 Task 15

Report the five-number summary

Goal: Use the `summary()` function to get a quick overview of one numerical measurement from the `penguins` dataset.

14.1.1 Step 1

Choose one numerical measurement variable from the `penguins` dataset.

You may use the same numerical variable you used in earlier Measures of Spread tasks, or you may choose another numerical measurement variable.

Good candidates are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`
- `body_mass_g`

Choose a measurement variable, not a categorical variable like `species`, `island`, or `sex`.

14.1.2 Step 2

Use `summary()` to get an overview of your chosen variable.

Replace `_____` with the column name you chose.

```
#  
summary(penguins$_____)
```

14.1.3 Step 3

Write a short interpretation of the summary output.

The `summary()` output helps you see the minimum, first quartile, median, mean, third quartile, and maximum. This gives you a quick picture of both center and spread.

Copy and paste this sentence pattern into your report. Replace each `_____` with the correct value or variable name.

```
**Comment:** In the `_____` column, the minimum value is _____, the median is _____, and the maximum value is _____.
```

Checks: Render to HTML.

After rendering, check that:

- the `summary()` output appears below your code chunk
- your interpretation uses values that match the `summary()` output
- your sentence is written as report text, not as a code comment

14.2 Task 16

End the Measures of Spread section with a takeaway

Goal: Finish the Measures of Spread section with one short takeaway about which measure of spread was most useful for your chosen variable.

14.2.1 Step 1

Choose the measure of spread that seemed most useful for your variable.

You may choose:

- standard deviation
- range
- interquartile range
- five-number summary

14.2.2 Step 2

Write one short takeaway.

Copy and paste this sentence pattern into your report. Replace each `_____` with your own words.

```
**Reflection:** For the `_____` variable, I think the most useful measure of spread is _____. I chose _____.
```

14.2.3 Step 3

Checks: Render to HTML.

After rendering, check that:

- your Measures of Spread section has all four subsections
- your final takeaway appears after the Five-Number Summary subsection
- your report text is readable and not only a list of code outputs

You have finished all sections of the Data Understanding Notebook! Congratulations on building a report that shows your code, your reasoning, and your interpretations.

15 Summary

16 Summary

In this activity, you were not only learning how to calculate descriptive statistics. You were practicing formatting Quarto reports.

You began by identifying variable types. Then you used that information to choose appropriate count-based summaries, measures of center, and measures of spread. As you built the notebook, you practiced explaining your code, reporting results in plain language, and reflecting on why a method does or does not fit a given variable.

You should now have a reproducible **Data Understanding Notebook** that shows both your code and your reasoning. You also practiced several Quarto report-building skills, including headings, YAML settings, inline R code, math text, callout notes, section labels, section links, and Markdown lists.

The bigger habit is this: slow down before summarizing data. Inspect the data first, check the variable type, choose a method that fits, and make sure your interpretation matches the evidence. These habits support credible, careful data analysis.

17 Chapter Terms

Callout Block: A formatted block in Quarto Markdown that sets off important content—such as a note, tip, warning, or reflection—so it stands out visually in the rendered document. Callout blocks are created with `:::` syntax and a callout class such as `.callout-note`, and in HTML output they appear as styled boxes with distinctive formatting.

Categorical Data: Data whose values place observations into groups or categories. The categories may have no natural order (nominal) or may have an order with uneven gaps between categories (ordinal). In R, categorical data are often stored as factors.

Census: A complete count or measurement of every member of a population, rather than just a subset.

Central Tendency: A measure that represents the center or typical value of a dataset. Common measures of central tendency include the mean, median, and mode.

Continuous Data: Numerical data that can take any value within a range, including values between other values. Measurements such as height, weight, and time are typical examples.

Descriptive statistics: Numerical summaries that describe the main features of a dataset, such as measures of central tendency (mean, median, mode) and measures of variability (range, variance, standard deviation).

Discrete Data: Numerical data that can take only specific, separate values, often counts such as 0, 1, 2, 3, and so on. In practice, discrete data are commonly integer-valued.

Frequency: The number of times a value or category appears in a dataset.

Inline Code: Code-style text placed within a sentence by surrounding the text with single backticks, such as `penguins`. In Quarto and Markdown, inline code is used to mark object names, function names, file

names, or short code expressions so they appear in a monospace code font in the rendered report. This formatting signals to the reader, “treat this as code or a code-related name,” not as ordinary prose.

Inline R Code: A short R expression written inside backticks with the prefix `r` (for example, ``r Sys.Date()``) that is evaluated when you knit/render the document and replaced with its result in the output. Inline R is used to insert dynamic values (dates, computed statistics, labels) directly into your narrative text, and it can also be used in YAML fields like date: “``r Sys.Date()``” when supported.

Interquartile Range (IQR): The range of the middle 50% of the data, calculated as $Q3 - Q1$. Because it ignores the lowest 25% and highest 25%, it is less sensitive to extreme values than the full range.

Length: The number of elements in an object. For a vector, `length(x)` is the count of elements in the vector; for a list, it’s the number of list components; for a data frame, it returns the number of columns.

Mean: The arithmetic average of a set of numbers, found by adding all values and dividing by the number of values.

Median: The middle value in an ordered dataset. If there is an even number of values, the median is the average of the two middle values.

Mode (statistic): The value or category with the greatest frequency in a dataset. A dataset can have one mode, more than one mode, or no mode if no value occurs more often than the others.

Numerical Data: Data recorded as numbers, where the values have meaningful magnitudes and consistent intervals. Numerical data are also called *quantitative* data.

Ordered Sample: A sample in which the order of the selected elements does matter, so (1, 2, 3) is different from (3, 2, 1). In counting language, this is the idea behind a *permutation*.

Ordinal Data: Categorical data with a meaningful ranking or order, but where the differences between adjacent categories are not necessarily equal or measurable. Examples include Likert-scale responses such as “strongly disagree” to “strongly agree.”

Parameter: A numerical summary of a population.

Percentile: A value that marks the point at or below which a given percentage of observations fall. For example, the 50th percentile is the median.

Population: The entire collection of individuals, cases, or objects you want to study.

Quartile: One of the cut points that divides ordered data into four equal parts. The first quartile (Q1) is the 25th percentile, the second quartile (Q2) is the median, and the third quartile (Q3) is the 75th percentile.

Range: A simple measure of spread found by subtracting the smallest value from the largest value in a dataset.

Sample: A subset of the population that you actually observe or measure.

Source: The plain-text editing view for an `.Rmd` or `.qmd` file, where you can see and edit the actual document syntax, including the YAML header, Markdown formatting, code chunk fences, and inline R. Use Source when instructions ask you to type exact code or formatting. In RStudio, you can switch to this view with the Source button at the top-left of the document toolbar.

Spread: How much the values in a dataset vary or scatter rather than cluster tightly together. Common measures of spread include range, interquartile range (IQR), variance, and standard deviation.

Standard Deviation: A measure of spread equal to the square root of the variance. Because it takes the square root, standard deviation is expressed in the same units as the original data.

Statistic: A numerical summary computed from a sample.

Summation: The process of adding a sequence of values together. In formulas, summation is written with the capital Greek letter sigma, Σ , as a compact way to show a sum.

Unordered Sample: A sample in which the order of the selected elements does not matter, so $\{1, 2, 3\}$ represents the same selection as $\{3, 2, 1\}$. In counting language, this is the idea behind a *combination*.

Variance: A measure of spread that describes how far values tend to vary from the mean, based on squared deviations. In introductory statistics, sample variance is commonly calculated using $n - 1$ in the denominator.

Visual: The formatted editing view for an `.Rmd` or `.qmd` file, where the document appears more like a word processor while still saving as Markdown underneath. Visual is useful for writing narrative text, tables, citations, and formatted content, but it can hide some of the syntax students need to learn. In RStudio, you can toggle between Source and Visual using the buttons at the top-left of the document toolbar.